

OpenBid — A Concurrent Online Auction House in Java

CS-GY 6103 · Introduction to Java · Final Project Report

Divyansh Maurya

Net ID: dm6602 · N-Number: N11376464

Department of Computer Science and Engineering

New York University, Tandon School of Engineering

Abstract—OpenBid is a multi-user online auction application built entirely in Java that lets many users list items, browse live auctions, and bid against one another in real time while an independent countdown runs on each lot. The application deliberately combines four advanced topics from the second half of the course so that they depend on one another rather than sitting side by side: a Java Swing desktop client, a TCP client/server protocol that pushes updates to every connected client, a multithreaded server built on a fixed thread pool with a single synchronized serialization point for all auction mutations, and a JDBC/SQLite persistence layer whose bid-accept and auction-close paths run as explicit ACID transactions guarded by an optimistic version column. The central engineering problem is competitive access to shared data: when two users bid on the same item in the same instant, exactly one must win, the loser must be told immediately with the new price, every observer must converge to the same current price, and the sale must be recorded fully or not at all. A scripted stress-test client that fires 240 concurrent bids from twelve threads confirms these guarantees empirically, and a 26-method JUnit 5 suite exercises the transactional, anti-snipe, proxy-bidding, reserve, and buy-it-now paths. Beyond the proposed core, the delivered system adds proxy (automatic) bidding, anti-sniping timer extension, reserve prices, Buy-It-Now, per-lot image catalogs, watch lists, and live outbid notifications.

Index Terms—Java, Swing, TCP sockets, client/server, multithreading, thread pool, concurrency, synchronization, JDBC, SQLite, ACID transactions, optimistic locking, PBKDF2, online auction.

I. INTRODUCTION

The idea for OpenBid came from a simple question about sites such as eBay: how does an auction guarantee that two people bidding in the final second cannot both win the same item? That question is not really about the user interface; it is about what happens on the server when many people compete for the same piece of data at the same moment. Building a small but working auction house is a natural way to answer it, because a correct auction needs every major topic from the second half of the course in one place and, more importantly, needs those topics to cooperate.

An auction is interesting precisely because its users are adversarial over shared state. Two bidders will often try to raise the price on the same lot at almost the same instant. The server has to ensure that only one of those bids succeeds, that the losing bidder learns immediately that they were outbid and what the new price is, that everyone watching the lot converges on the same current price without refreshing, and that when the clock expires the sale is written to the database completely or not at all. Multi-user concurrency and transactional atomicity are therefore the core of the design, not a feature bolted on at the end.

This report describes the delivered system against the four required advanced concepts, explains the concurrency and transaction design that makes the bidding path correct, and reports the results of a concurrent stress test and a JUnit test suite. The delivered application also went beyond the original proposal, adding proxy bidding, anti-sniping, reserve prices, Buy-It-Now, image catalogs, watch lists, and targeted outbid notifications.

A. Advanced Concepts Used

The grading criterion requires at least three advanced Java concepts; OpenBid uses four, each load-bearing rather than decorative:

- **Graphical user interfaces.** the entire client is a Java Swing desktop application — login and registration, a sortable

auction table, a live bidding panel with a scrolling bid feed, and a listing dialog.

- **Networking.** the client and server communicate over TCP sockets with a compact, escaped text protocol; the server pushes events so the bid feed is genuinely real-time rather than polled.
- **Multithreading and concurrency.** the server hands each connection to a fixed pool of handler threads, funnels every auction mutation through one synchronized manager, and runs per-lot countdowns on a scheduled executor.
- **Databases and transactions.** users, auctions, bids, sales, proxy bids, images, and watches live in SQLite through JDBC, with the money-changing operations run as explicit commit/rollback transactions.

A fifth, security, is included as a bonus: passwords are never stored in plaintext but hashed with PBKDF2-HMAC-SHA256 using a random per-user salt and an application-level pepper.

II. SYSTEM ARCHITECTURE

OpenBid follows a classic three-tier client/server structure, shown in Fig. 1. The presentation tier is a Swing client; the logic tier is a socket server with a thread pool and a single serialization point; the persistence tier is a set of DAO classes over one SQLite database. A shared package defines the wire protocol and the immutable data-transfer records used by both sides, so the client and server never disagree about message formats.

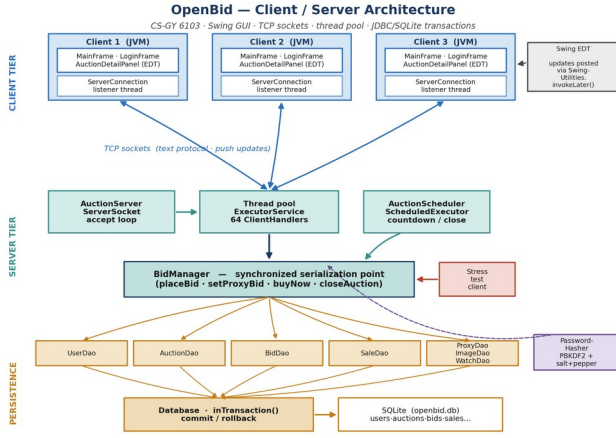


Fig. 1. Three-tier architecture. Swing clients exchange text messages with the server over TCP; the accept loop hands each connection to a 64-thread pool of handlers; every auction mutation is serialized through the BidManager and committed to SQLite through the DAO layer. The scheduler closes lots on their own threads through the same monitor.

The code is organized into six Java packages under `com.openbid`, summarized in Table I. The split keeps the responsibilities clean: the shared package has no dependency on either side, the client never touches JDBC, and the server never touches Swing.

TABLE I. Source Packages and Responsibilities

Package	Responsibility
shared	Wire protocol, AuctionInfo/BidInfo records, Money, Categories, catalog images
client	Swing UI: LoginFrame, MainFrame, AuctionDetailPanel, ListItemDialog, ServerConnection
server	AuctionServer, ClientHandler, BidManager, AuctionScheduler, result records
db	Database + <code>inTransaction()</code> ; UserDao, AuctionDao, BidDao, SaleDao, ProxyBidDao, ImageDao, WatchDao
security	PasswordHasher (PBKDF2 + salt + pepper)
tools	StressTestClient — scripted concurrent-bid harness

The system is built with Maven and the shade plugin produces a single runnable server JAR; the client is launched from the same build. The project comprises roughly 5,000 lines of application Java across 37 source files, plus a JUnit 5 test module.

III. NETWORKING

The client and server speak a line-oriented text protocol over TCP. Each message is a single line of fields separated by the pipe character, with backslash, pipe, and newline escaped so that free-text titles and descriptions can contain any character without breaking the framing. The Protocol class in the shared package is the single source of truth for message names and for the encode/decode routines, so both sides parse messages identically.

Requests flow from client to server (REGISTER, LOGIN, LIST_ITEM, BID, PROXY_BID, BUY_NOW, WATCH, and the various GET queries). Crucially, the server also emits unsolicited events that it pushes to clients: NEW_BID and AUCTION_UPDATED broadcast a price change to everyone, AUCTION_EXTENDED signals an anti-snipe extension, AUCTION_CLOSED announces a result, and targeted messages

such as YOU_OUTBID and YOU_WON go to a single user. This push model is what makes the bid feed update instantly instead of the client polling in a loop.

TABLE II. Representative Protocol Messages

Direction	Message	Purpose
C → S	BID id amount	Place a manual bid on a lot
C → S	PROXY_BID id max	Set a hidden maximum for auto-bidding
C → S	BUY_NOW id	Purchase immediately at the fixed price
S → C	NEW_BID ...	Broadcast: a new bid was accepted
S → C	AUCTION_UPDATED ...	Broadcast: current price/leader changed
S → C	AUCTION_EXTENDED id end	Broadcast: anti-snipe extended the clock
S → C	YOU_OUTBID id ...	Targeted: you are no longer the leader
S → C	AUCTION_CLOSED id winner ...	Broadcast: lot closed, sale recorded

On the client, a dedicated ServerConnection owns the socket and runs a single listener thread. Because Swing components may only be touched on the Event Dispatch Thread, the listener never mutates the UI directly; every inbound line is handed to the EDT with `SwingUtilities.invokeLater`. On the server, each ClientHandler synchronizes its outbound writes so that a direct reply and a broadcast cannot interleave into a corrupted line. Sockets use `TCP_NODELAY` to keep bid latency low.

IV. MULTITHREADING AND CONCURRENCY

Concurrency is the heart of the project. Three kinds of threads run on the server at once, and the design's job is to let them serve many users in parallel while still making every auction mutation behave as if it happened one at a time.

A. Threads on the Server

- **Accept thread.** the main accept loop blocks on `ServerSocket.accept` and, for each new connection, constructs a `ClientHandler` and submits it to the pool.
- **Handler pool.** a fixed `ExecutorService` of 64 daemon threads runs the `ClientHandlers`. Each handler reads requests from its own socket, so slow clients never block one another.
- **Scheduler thread.** a single-threaded `ScheduledExecutorService` arms one delayed close task per open lot; anti-snipe cancels and re-arms it.

B. One Serialization Point

Every operation that changes auction state — placing a bid, setting a proxy maximum, buying now, and closing a lot — is a synchronized method on a single shared `BidManager` instance. Because all of these methods lock the same monitor, two last-second bids on the same item are processed strictly one after the other: the first to acquire the lock runs to completion (record the bid, raise the price, broadcast), and only then can the second proceed and be rejected against the already-updated price. The same lock is taken by the scheduler's close task, so a lot cannot expire in the middle of a bid that is still in flight. Serializing all mutations on one monitor also matches SQLite's single-writer model, avoiding lock contention at the database.

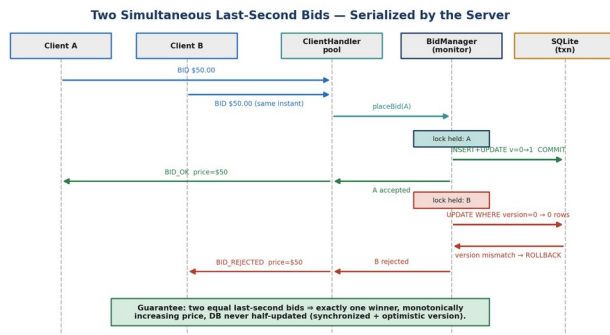


Fig. 2. Two equal last-second bids. The handler pool funnels both into the BidManager monitor; the first commits and wins, the second finds a stale version and is rolled back and rejected — exactly one winner, monotonically increasing price.

C. Proving It Under Load

To show the guarantee empirically rather than by assertion, the tools package contains a `StressTestClient`. It registers twelve bidders, opens one lot, and then releases all twelve threads on a countdown latch so they fire simultaneously — 20 bids each, 240 in total. After the storm it reopens the database directly and checks four invariants: the number of accepted bids equals the number of rows recorded, bid amounts are strictly increasing, the stored current price equals the top bid, and the recorded leader is the top bidder. The harness prints PASS only when every check holds and accepted + rejected equals the number fired.

V. DATABASES AND TRANSACTIONS

Persistence uses SQLite through JDBC behind a thin DAO layer. A single Database object owns the one JDBC connection and the transaction boundary; the schema is applied from a bundled `schema.sql` on first run, with lightweight in-code migrations that add later columns to existing databases. The relational design is shown in Fig. 3.

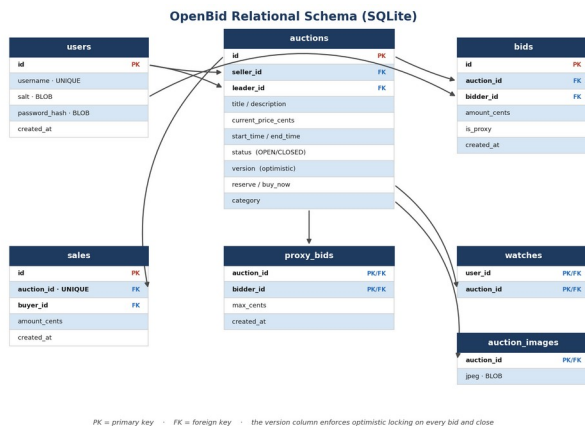


Fig. 3. Relational schema. The auctions table carries the leader, current price, status, and an optimistic version column; bids, sales, proxy_bids, watches, and auction_images reference it by foreign key.

A. Transactional Bidding and Closing

The money-changing operations are explicit transactions. The `Database.inTransaction` helper turns auto-commit off, runs the supplied work, commits on success, and rolls back on any exception before rethrowing. Accepting a bid inserts the bid row and updates the auction's price, leader, and bid count inside one

transaction, so the database can never end up with a recorded bid but a stale price. Closing a lot marks it CLOSED and, if it sold, inserts the sale row in the same transaction — the unique constraint on `sales.auction_id` guarantees a lot can sell at most once even if the close path runs twice.

B. Optimistic Locking as a Second Guard

On top of the server-side synchronization, the auctions table carries an integer version column. The bid update is written as `UPDATE ... WHERE id = ? AND status = 'OPEN' AND version = ?`, incrementing the version on success. If a concurrent change slipped in, the row count comes back zero, the code raises an `OptimisticLockException`, the transaction rolls back, and the bidder is told to try again against the fresh price. In normal operation the single monitor already prevents races, but the version check makes the correctness independent of that assumption and is exactly the kind of lost-update defense a production auction would keep. A JUnit test forces a mid-transaction version mismatch and confirms the inserted bid row is rolled back and no phantom bid remains.

VI. GRAPHICAL USER INTERFACE

The client is a Swing application. After launching, a user registers or logs in through a `LoginFrame`; the `MainFrame` then shows all lots in a sortable `JTable` with the item, current price, leader, category, and time remaining, refreshed live as server events arrive. Selecting a lot opens the `AuctionDetailPanel`, which shows the item image, a scrolling bid history, and controls to bid, set a proxy maximum, buy now, or add the lot to a watch list. A `ListItemDialog` lets any user act as a seller, supplying a title, description, starting price, category, optional reserve and Buy-It-Now prices, duration, and an optional JPEG image.

Responsiveness is a correctness property here, not just polish: since every server update is marshalled onto the EDT, the interface keeps repainting countdowns and prices smoothly even while a burst of bid events is arriving. A lightweight Swing timer ticks the per-lot countdowns once a second without any server round-trip.

VII. SECURITY

Passwords are never stored or transmitted in a recoverable form on disk. At registration the server draws a random 16-byte per-user salt, appends an application-level pepper read from the environment, and derives a 256-bit key with PBKDF2-HMAC-SHA256 over 210,000 iterations. Only the salt and derived hash are stored. Verification recomputes the hash and compares it in constant time with `MessageDigest.isEqual` to avoid timing leaks. Usernames are validated against a strict pattern and passwords carry a minimum length. This satisfies the recommendation to salt-and-pepper the credential table and keeps the auction's identity of leaders and winners trustworthy.

VIII. EXTENDED FEATURES BEYOND THE PROPOSAL

With the required core complete, several stretch goals from the proposal were delivered, plus a few extras that make the auction feel real:

- **Anti-sniping.** a bid inside the final 30 seconds pushes the end time out to 30 seconds from now, cancels and re-arms the

scheduler, and broadcasts AUCTION_EXTENDED, so a lot cannot be stolen in the last instant.

- **Proxy (automatic) bidding.** a user sets a hidden maximum and the server bids on their behalf, jumping straight to the deciding amount one increment above the runner-up. The resolution loop is re-entrant under the same monitor, so proxy wars settle atomically.
- **Reserve prices.** a seller may set a hidden reserve; a lot that ends below it closes unsold rather than transferring at too low a price.
- **Buy-It-Now.** an optional fixed price lets a buyer end the auction immediately; the purchase and close commit together.
- **Image catalog.** each lot carries a JPEG stored as a BLOB (a generated placeholder when none is supplied); reads take the same monitor as writes so a handler never reads a half-written image.
- **Watch lists and notifications.** users can watch lots and receive a targeted WATCH_ALERT when a watched lot moves, and the previous leader receives a YOU_OUTBID message the moment they are overtaken.

IX. TESTING AND EVALUATION

The project is evaluated along the four axes set out in the proposal — concurrency, atomicity, functionality, and usability — using both an automated JUnit 5 suite and the concurrent stress harness.

A. Unit and Integration Tests

The test module contains 26 JUnit 5 test methods across four classes, run headless against an in-memory SQLite database so they are fast and deterministic. Table III summarizes them. The BidManager tests alone cover 20 behaviors, including the first-bid-at-starting-price rule, too-low and self-outbid rejection, anti-snipe extension, sale recording, idempotent double-close, reserve-not-met, Buy-It-Now, proxy resolution, relisting, and the version-mismatch rollback.

TABLE III. JUnit 5 Test Coverage

Test class	Tests	Focus
BidManagerTest	20	Bidding, proxy, anti-snipe, reserve, buy-now, close, rollback
PasswordHasherTest	2	PBKDF2 hash/verify and wrong-password rejection
DatabaseTransactionTest	2	Commit persists; rollback discards
ProtocolTest	2	Encode/decode round-trip and escaping

B. Concurrency and Atomicity Results

Running the StressTestClient against a live server fires 240 simultaneous bids from twelve threads. In every run the server serializes them correctly: the count of accepted bids equals the rows recorded, the amounts are strictly increasing, the stored price matches the top bid, and the single recorded leader is the top bidder — the harness reports PASS. Atomicity is verified separately by the version-mismatch test, which forces a failure mid-transaction and confirms the database rolls back to a consistent state with no orphaned bid row. Together these satisfy the concurrency and atomicity evaluation criteria, while the JUnit functional tests and the responsive live demo cover functionality and usability.

X. CONCLUSION

OpenBid brings Swing, TCP sockets, a multithreaded server, and JDBC transactions together in one application where the pieces genuinely depend on one another. The bidding path is the thread that ties them: concurrent requests arrive over the network, are serialized by a single synchronized manager on a thread pool, are committed atomically to SQLite under an optimistic version guard, and the result is pushed back to every Swing client on the Event Dispatch Thread. The project meets the requirement of at least three advanced concepts with four, adds password security, and delivers the proposal's stretch goals — anti-sniping and proxy bidding — along with reserves, Buy-It-Now, image catalogs, and watch lists. A concurrent stress test and a 26-method unit suite give empirical confidence that the central promise of an auction house holds: when two people bid in the same instant, exactly one wins and the books always balance.

REFERENCES

- [1] C. S. Horstmann, Core Java, Volume I & II, 12th ed. Boston, MA, USA: Pearson, 2021.
- [2] Oracle, "The Java Tutorials: Concurrency," Oracle Corp. [Online]. Available: <https://docs.oracle.com/javase/tutorial/essential/concurrency/>
- [3] Oracle, "Creating a GUI With Swing," The Java Tutorials. [Online]. Available: <https://docs.oracle.com/javase/tutorial/uiswing/>
- [4] Oracle, "java.util.concurrent — ExecutorService and ScheduledExecutorService," Java SE 17 API Specification, 2021.
- [5] SQLite Consortium, "SQLite: Atomic Commit and Write-Ahead Logging," [Online]. Available: <https://www.sqlite.org/atomiccommit.html>
- [6] Xerial, "sqlite-jdbc: SQLite JDBC Driver," [Online]. Available: <https://github.com/xerial/sqlite-jdbc>
- [7] B. Kaliski, "PKCS #5: Password-Based Cryptography Specification Version 2.0," RFC 2898, IETF, Sep. 2000.
- [8] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002 (Data Access Object and Optimistic Offline Lock).